Week 6 — Neural Network Pricer for American Puts

Aarav | Options Pricing Course | Training an MLP to match CRR binomial American put prices

Part A — Dataset Generation

12,000 synthetic American put contracts were sampled uniformly over the ranges below and labeled with the Week 4 CRR binomial pricer (crr_put_price) at a fixed 200-step tree, so every label uses the same discretization error.

Feature	Range	Rationale
S0 (spot)	U[50, 150]	generic equity price scale
K (strike)	U[50, 150]	same scale as S0 -> moneyness spans ~0.33x-3.0x
T (maturity, yrs)	U[0.05, 2.0]	2 weeks to 2 years
r (risk-free rate)	U[0.00, 0.10]	0%-10% annual, continuously compounded
sigma (volatility)	U[0.05, 0.60]	5% low-vol blue chip to 60% high-vol growth stock

Label sanity checks (all passed on 12,000/12,000 rows):

- Finite: no NaN/Inf prices.
- Non-negative: no price below 0.
- At least intrinsic value: no price below max(K - S0, 0).

The labeled dataset is saved to data/american_put_dataset.csv so training can be rerun without regenerating labels (label generation takes ~40s for 12,000 contracts at 200 steps).

Part B — Training

The dataset was split 80/10/10 into train/validation/test with a fixed seed (42). Features were standardized (z-score) using training-set mean/std only, then applied unchanged to validation and test to avoid leakage.

Architecture: a PyTorch MLP, 5 -> 64 -> 64 -> 32 -> 1 (three hidden layers, ReLU activations), trained with Adam (lr=1e-3), MSE loss, batch size 128, for up to 200 epochs with early stopping (patience 25 epochs on validation MSE). The checkpoint with the lowest validation MSE was saved and used for all downstream evaluation — not simply the final epoch.

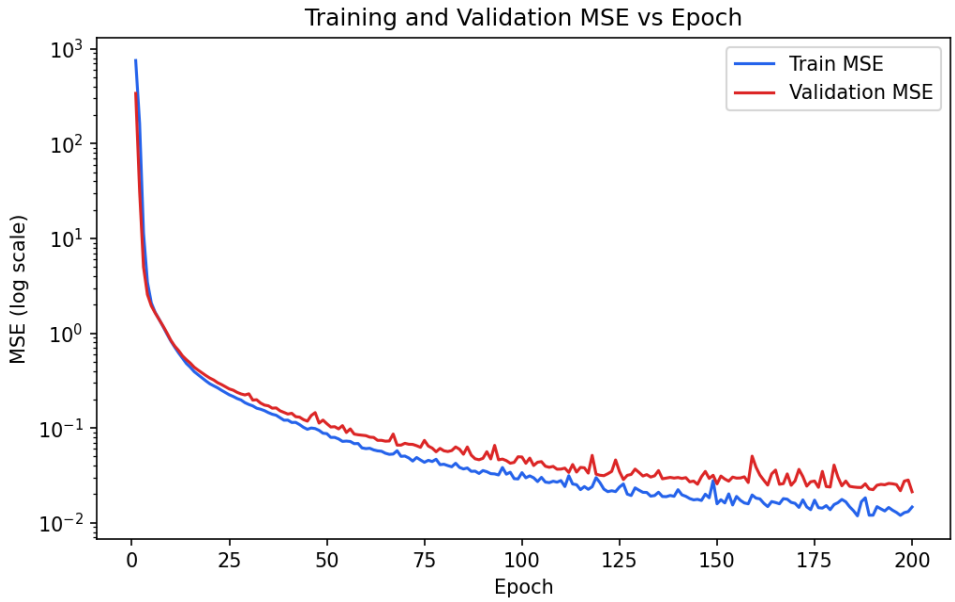

Figure 1. Train/validation MSE vs epoch (log scale). Both curves fall smoothly and track each other closely, with no sign of overfitting divergence; the best validation MSE was 0.0212 (val RMSE ≈ 0.15), reached at the final epoch (200) before early stopping would have triggered.

Part C — Evaluation and Finance Checks

Test-set error metrics (1,200 held-out contracts, never seen in training or model selection):

Metric	Value
MAE	0.0883
RMSE	0.1230
Max absolute error	1.0538

MAE by moneyness bucket (moneyness = S0/K; puts are ITM when S0 < K):

Bucket	MAE	n (test)
Deep ITM (S0/K < 0.85)	0.0967	419
Near ATM (0.85-1.15)	0.1140	332
Deep OTM (S0/K > 1.15)	0.0614	449

Near-ATM contracts have the highest MAE (0.114). This is expected: near-the-money American puts have the most curvature in price with respect to S0 (highest gamma), so a fixed-capacity MLP has the hardest time fitting that region tightly.

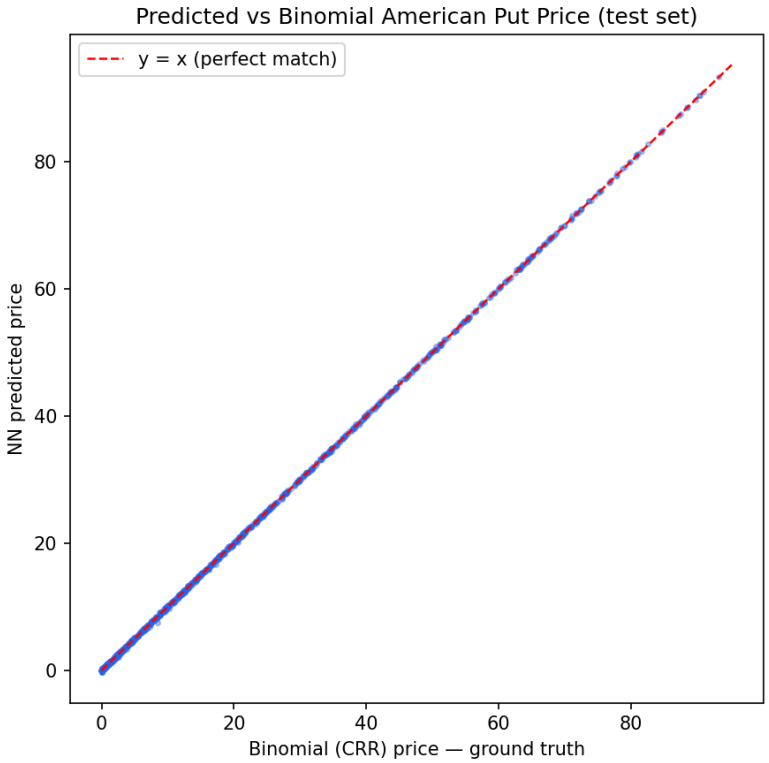

Figure 2. Predicted (NN) vs ground-truth (binomial) price on the test set. Points hug the y=x line closely across the full $0-$100 price range, with the visible scatter concentrated at low prices (near-zero OTM contracts) and mild widening at higher prices.

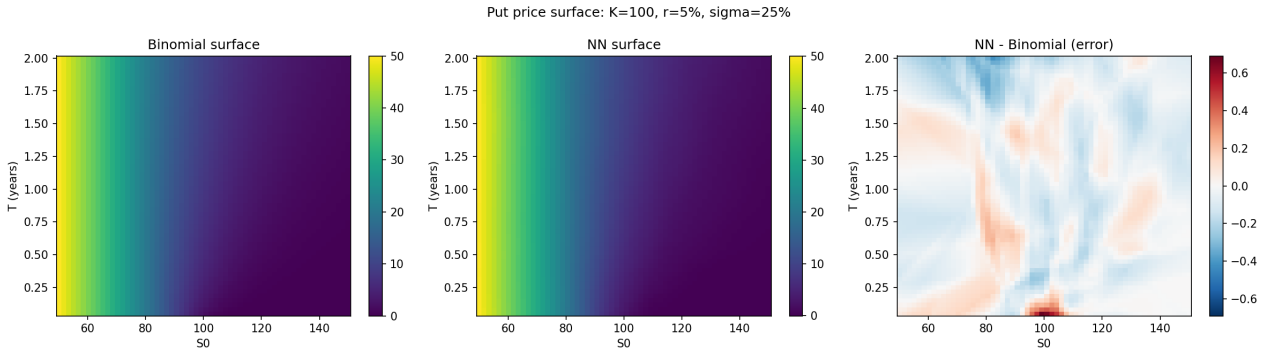

Figure 3. Put price surface for fixed K=100, r=5%, sigma=25%, varying S0 and T. The binomial and NN surfaces (left, middle) are visually indistinguishable; the error surface (right) shows deviations concentrated at long maturities and low S0, on the order of a few cents to ~$0.30, well below the surface's own price scale (up to ~$50).

Finance sanity checks

- Non-negativity: 119/1,200 test predictions (≈10%) and 40/3,600 surface-grid points were slightly negative (worst: -$0.33). All violations occur for deep-OTM, low-volatility, short-to-medium-maturity puts whose true price is essentially $0 — the unconstrained linear output layer has no floor at zero, so it can dip slightly below it. This is a real limitation, not a rounding artifact.
- Monotonicity in S0: checked the NN price along S0 = 50 to 150 (K=100, T=1, r=5%, sigma=25%, 199 steps). 0/199 steps increased — the learned price is strictly non-increasing in spot, matching the no-arbitrage property of a put.
- Intrinsic value floor: 8/1,200 test predictions fell more than $0.25 below intrinsic value max(K-S0, 0) (worst breach $1.05, on a deep-ITM, low-vol, long-maturity contract). These are true violations of the American-put early-exercise floor and represent the sharpest failure mode found.

Reflection: where does the model perform worst, and why?

Average error looks small (MAE $0.09 against prices up to ~$100), but averages hide two concrete weak regions:

- Near-zero-price deep-OTM puts (low vol, short T): the model occasionally predicts small negative prices here. The MSE loss barely penalizes a few-cent miss on a $0.00 label, so the network has little incentive to clamp at zero. This is the majority of the non-negativity violations and would be easy to fix with a ReLU/softplus output layer or a post-hoc clip.
- High-price, high-curvature contracts (deep ITM or near ATM, longer T, moderate-to-high sigma): this is where the single worst error ($1.05) and most of the intrinsic-floor breaches occur. These contracts have the steepest price surface and the largest absolute price scale, so a fixed absolute error budget from MSE training is spent disproportionately on the many cheap OTM contracts rather than the few expensive ITM ones.

Overall the model is financially sensible but not exact: it strictly respects monotonicity in S0, and its surface matches the binomial surface within a few cents almost everywhere, but it is not safe to deploy unclamped — a small fraction of quotes would violate the zero-price and intrinsic-value floors that any real put must satisfy. A production version should either clip outputs to max(intrinsic, 0) post-hoc, use a non-negative output activation, or add a no-arbitrage penalty term to the loss.

Reproducibility

Code and repo link: [insert your GitHub/Drive link here before submitting]. Run order from a clean environment:

```
pip install torch numpy pandas scikit-learn matplotlib reportlab
python generate_dataset.py  # Part A: builds data/american_put_dataset.csv
python train_mlp.py       # Part B: trains MLP, saves best checkpoint
python evaluate.py        # Part C: metrics, scatter, surface, sanity checks
python plot_learning_curve.py
```

All randomness (data sampling, train/val/test split, PyTorch weight init, DataLoader shuffling) is controlled by a single seed (42). Files produced: data/american_put_dataset.csv, data/test_split.csv, checkpoints/best_mlp.pt, checkpoints/scaler.json, checkpoints/history.json, checkpoints/test_metrics.json, checkpoints/sanity_results.json, and the four plots in plots/.